FIBONACCI SERIES IN DATA STRUCTURES

Case Study Assignment

Course: Data Science

Unit: Data Structures

Group Members:

1. Jill Daniel - CT204/124354/25.
2. Michelle Mutheu - CT204/124263/25
3. Lydiah Watutwe - CT204/125694/25
4. Aaron Kimutai - CT204/124320/25

Submission Date: 16 October 2026

1. Introduction

The Fibonacci sequence is a series of numbers in which each number is obtained by adding the two preceding numbers. The sequence begins with 0 and 1, followed by 1, 2, 3, 5, 8, 13, and so on.

In data structures and algorithms, the Fibonacci sequence is useful for studying different approaches to solving problems, including recursion, iteration, and dynamic programming.

This case study explores these approaches and compares their efficiency. It also applies the Fibonacci sequence to a customer loyalty reward system in which reward points grow according to the sequence.

2. Recursive Approach

2.1 Explanation of the Recursive Approach

Recursion is a programming technique in which a function calls itself to solve a problem. In the Fibonacci sequence, the function calculates a number by adding the two preceding Fibonacci numbers.

The function uses two base cases: when `n = 0`, it returns 0, and when `n = 1`, it returns 1. These base cases stop the function from calling itself indefinitely.

For example:

F(5) = F(4) + F(3) = 3 + 2 = 5.

2.2 Complexity Analysis

Time Complexity: $O(2^n)$

The recursive approach has exponential time complexity because it repeatedly calculates the same Fibonacci values. As the input increases, the number of function calls grows rapidly, making the method inefficient for large values of `n`.

Space Complexity: O(n)

The function uses memory for recursive calls on the call stack. The maximum depth of these calls grows proportionally to `n`.

2.3 Limitation

The main disadvantage of the recursive approach is that it repeats calculations unnecessarily. Consequently, its execution becomes slower as the Fibonacci position increases.

3. Iterative Approach Using an Array

3.1 Implementation

The iterative approach uses loops to generate Fibonacci numbers one after another. An array stores the generated numbers so that previously calculated values can be reused.

The program initializes the first two Fibonacci numbers, 0 and 1. It then uses a loop to calculate each subsequent number by adding the two preceding values.

C++ Program:

```
#include <iostream>

using namespace std;


int main() {

    int n = 6;

    int fib[6];


    fib[0] = 0;

    fib[1] = 1;


    for (int i = 2; i < n; i++) {

        fib[i] = fib[i - 1] + fib[i - 2];

    }


    for (int i = 0; i < n; i++) {

        cout << fib[i] << " ";

    }


    return 0;

}
```

Output:

0 1 1 2 3 5

3.2 Explanation

The array stores six Fibonacci values. The first two positions are initialized to 0 and 1. A `for` loop starts at index 2 and calculates each remaining value using the two previous array elements. A second loop displays all the values.

3.3 Efficiency

The iterative approach has **O(n) time complexity** because it calculates each Fibonacci value once. Its **space complexity is O(n)** because the array stores `n` values.

Unlike the simple recursive approach, iteration avoids repeatedly calculating the same Fibonacci numbers, making it more efficient for larger inputs.

4. Dynamic Programming Approach

4.1 Explanation of Dynamic Approach

Dynamic programming involves solving smaller problems once, storing their answers and reusing the answers instead of solving the same problems repeatedly.

4.2 Methods and their implementation:

4.2.1 Memoization: top-down

Uses recursion but stores calculated answers. If a value has already been calculated, the function reuses it.

4.2.2 Tabulation: bottom-up

Starts with the known values and fills an array from the smallest Fibonacci numbers to the largest.
For this case study, tabulation is suitable because the Fibonacci sequence is generated in order.

C++ program:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 6;
    vector<long long> dp(n);

    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i < n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    for (int i = 0; i < n; i++) {
        cout << dp[i] << " ";
    }

    return 0;
}
```

Output:
0 1 1 2 3 5

4.3 Time and space complexity

A. **Time complexity:**
   The loop calculates each fibonacci number once, so the amount of work grows linearly with the number of terms.
B. **Space Complexity:**
   The vector stores all *n* Fibonacci numbers, so the memory stored grows linearly with the number of terms.

4.4 Space-Time Trade-Off

Dynamic programming cuts down the time needed for calculations by saving results that have already been computed in memory. This does need storage space but it stops the same calculations from being repeated and makes the process faster.

With the tabulation approach both the time needed and the space needed grow at a rate of $O(n)$. This makes the approach good for creating a Fibonacci sequence, which the customer loyalty reward system needs.

Memoization also reaches $O(n)$ time and $O(n)$ space, for Fibonacci. It stores the cached results. Keeps the recursion stack in memory.

5.Application:Customer Loyalty Reward System.

5.1 Description

A business can use the Fibonacci sequence to demonstrate a loyalty reward system. The custom enters the number of years they have remained loyal ,and the program assigns points for each year according to the Fibonacci sequence. In this example, the first years 1 point, the second year earns 1 point , and each later year earns the sum of the pointts for the two preceding years.The points are stored in an array and displayed in a table .

5.2 C++ Implementation

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int years;
    cout << "Enter number of loyalty years (1-90): ";
    cin >> years;

    if (years < 1 || years > 90) {
    cout << "Please enter a number from 1 to 90." << endl;
    return 1;
    }

    vector<long long> points(years + 1);
     points[1] = 1;

    if (years >= 2) {
    points[2] = 1;
    }

    for (int year = 3; year <= years; year++) {
    points[year] = points[year - 1] + points[year - 2];
    }

    cout << "\nYear\tReward Points\n";
    cout << "----------------------\n";
    for (int year = 1; year <= years; year++) {
    cout << year << "\t" << points[year] << endl;
    }

    return 0;
}
```

5.3 Example Output

For an input of 6 years, the output is:

Loyalty Year	Reward Points
1	1
2	1
3	2
4	3
5	5
6	8

The program uses O(n) time to calculate the points and O(n) space to store them. The input is limited to 90 years because Fibonacci values grow quickly and values beyond this range can exceed the capacity of a signed 64-bit integer.

TASK 5: PERFORMANCE ANALYSIS OF FIBONACCI SERIES GENERATION METHODS

5.1 Introduction

Performance analysis is the process of evaluating how efficiently an algorithm executes in terms of execution time and memory usage. In this task, three approaches to generating the Fibonacci sequence are compared: recursion, iteration, and dynamic programming.

The experiment uses (n = 30), meaning that each algorithm generates the first 30 Fibonacci numbers. The objective is to determine which method performs best and recommend the most suitable approach for the company's customer loyalty reward system.

5.2 Python Program for Performance Analysis

The following Python program measures the execution time of each method using Python's "time" module.

```
import time

# Recursive approach
def fibonacci_recursive(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)


def recursive_series(n):
```

```
    return [fibonacci_recursive(i) for i in range(n)]


# Iterative approach
def fibonacci_iterative(n):
    fib = []

    for i in range(n):
        if i == 0:
            fib.append(0)
        elif i == 1:
            fib.append(1)
        else:
            fib.append(fib[i - 1] + fib[i - 2])

    return fib


# Dynamic programming approach
def fibonacci_dp(n):
    if n <= 0:
        return []

    dp = [0] * n

    if n >= 2:
        dp[1] = 1

    for i in range(2, n):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp


# Performance analysis
n = 30

methods = [
    ("Recursive", recursive_series),
    ("Iterative", fibonacci_iterative),
    ("Dynamic Programming", fibonacci_dp)
]

for name, method in methods:
    start = time.perf_counter()
    result = method(n)
    end = time.perf_counter()
```

```
    execution_time = end - start

    print(name, "execution time:",
          format(execution_time, ".8f"), "seconds")
    print("Last Fibonacci number:", result[-1])
    print()
```

5.3 Explanation of the Program

1. Importing the time module: The statement "import time" imports Python's time module, which provides functions for measuring execution time.

2. Implementing recursion: The recursive function calculates Fibonacci numbers by calling itself to calculate the two preceding values. It uses base cases for 0 and 1 to stop the recursion.

3. Implementing iteration: The iterative function uses a loop and an array to generate the Fibonacci sequence. Each new number is calculated by adding the previous two numbers.

4. Implementing dynamic programming: The dynamic programming function stores previously calculated values in an array. This avoids repeating calculations and improves efficiency.

5. Setting the input: The statement "n = 30" specifies that each algorithm should generate the first 30 Fibonacci numbers.

6. Recording the starting time: "time.perf_counter()" records a high-resolution timestamp immediately before an algorithm executes.

7. Executing the algorithm: "result = method(n)" runs the selected function and stores its output.

8. Recording the ending time: A second call to "time.perf_counter()" records the timestamp after execution.

9. Calculating execution time: The ending time is subtracted from the starting time to determine how long the algorithm took to execute.

10. Displaying the results: The program prints each method's execution time and the last Fibonacci number generated.

5.4 Performance Results

The following table illustrates sample measurements from one Python environment. Actual execution times may vary depending on the computer's hardware, Python version, and system workload.

Algorithm| Execution time (seconds)| Last Fibonacci number

Recursive| 0.20121767| 514229
Iterative| 0.00001093| 514229
Dynamic programming| 0.00000508| 514229

Note: These measurements are illustrative. The program should be executed on the group's computer to obtain the actual execution times for the final report.

5.5 Interpretation of Results

The results show that all three algorithms generate the same final Fibonacci number, 514229, when producing the first 30 terms.

However, their execution times differ significantly.

The recursive approach is the slowest because it recalculates the same Fibonacci numbers many times. This leads to a large number of function calls and unnecessary computational work.

The iterative approach is faster because it uses a loop to calculate each Fibonacci number once and stores the results in an array.

The dynamic programming approach is also efficient because it stores intermediate results and reuses them instead of recalculating them. Small timing differences between iteration and dynamic programming depend on the implementation and computer environment.

5.6 Time and Space Complexity

Algorithm| Time complexity| Space complexity
Recursive| $\Theta(\varphi^n)$| O(n)
Iterative| O(n)| O(n)
Dynamic programming| O(n)| O(n)

Here, φ (phi) is approximately 1.618.

The recursive approach has exponential time complexity because the number of function calls grows rapidly as the input increases. Its space complexity is O(n) because recursive calls occupy the call stack.

The iterative and dynamic programming approaches have linear time complexity because each Fibonacci number is calculated once. Their space complexity is O(n) because they store the generated sequence in an array.

5.7 Recommendation

Based on the performance analysis, the company should use the iterative approach with an array for its customer loyalty reward system.

This approach is recommended because:

1. It calculates each Fibonacci number only once.
2. It has linear time complexity, O(n).
3. It stores the reward points for each year in an array.
4. It avoids the repeated calculations associated with basic recursion.
5. It is simple to implement, test, maintain, and scale.

Dynamic programming using tabulation is also suitable because it has the same time and space complexity. In fact, array-based iterative generation is itself a form of bottom-up dynamic programming.

5.8 Conclusion

The performance analysis demonstrates that the recursive approach is less efficient than the iterative and dynamic programming approaches because it performs repeated calculations. The iterative and dynamic programming approaches generate the Fibonacci sequence more efficiently, with linear time complexity.

Therefore, the array-based iterative approach is recommended for the company's customer loyalty reward system because it provides efficient computation, convenient storage of yearly reward points, and good scalability as the number of customers and loyalty years increases.